Chapter-17

Farhan
1603084

Testability:-

(1) Operability:-

- The system has fewer bugs.
- No bugs block the execution of tests.

(2) Observability:-

- Distinct output is generated for each input.
- System states and variables are visible during execution.
- All factors affecting the output are visible.

(3) Controllability:-

- All possible outputs can be generated through some combination of input.
- All code is executable through some combination of input.

(4) Decomposability:-

- The software system is built from independent modules.
- Software modules can be tested independently.

(5) Simplicity:-

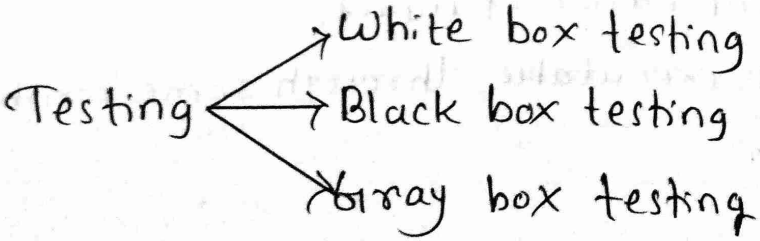
- Functional simplicity.
- Structural simplicity.
- Code simplicity.

(6) Stability:-

- Changes to the software are infrequent.
- Changes to the software are controlled.

(7) Understandability:-

- The design is well understood.
- The documentation is well organized.



↗ Glass box testing

White Box Testing:-

- White box testing is a test case design method that uses the control structure of the procedural design to derive test cases.

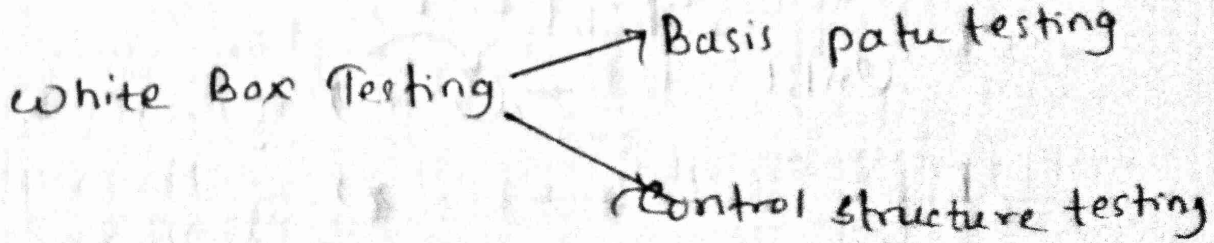
- Using white box testing, the soft. engineer can derive test cases that -

  (i) Guarantees that all independent paths within a module have been exercised at least once.

  (ii) Exercise all logical decisions on their true and false sides.

  (iii) Execute all loops at their boundaries and within their operational bounds.

  (iv) Exercise internal data structures to ensure their validity.

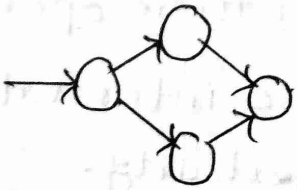


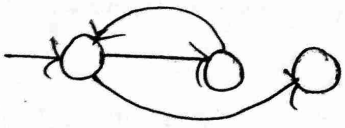
Basis Path Testing:-

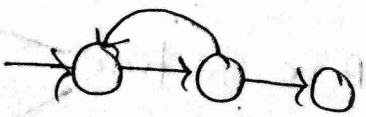
- Is a white-box testing technique.
- Enables the test case designer to to derive a logical complexity measure of a procedural design and use this measure as a guide for defining a basis set of execution paths.
- Guarantees to execute every statement in the program at least once.

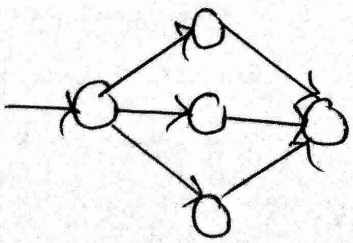
Flow graph notation:-

sequence 

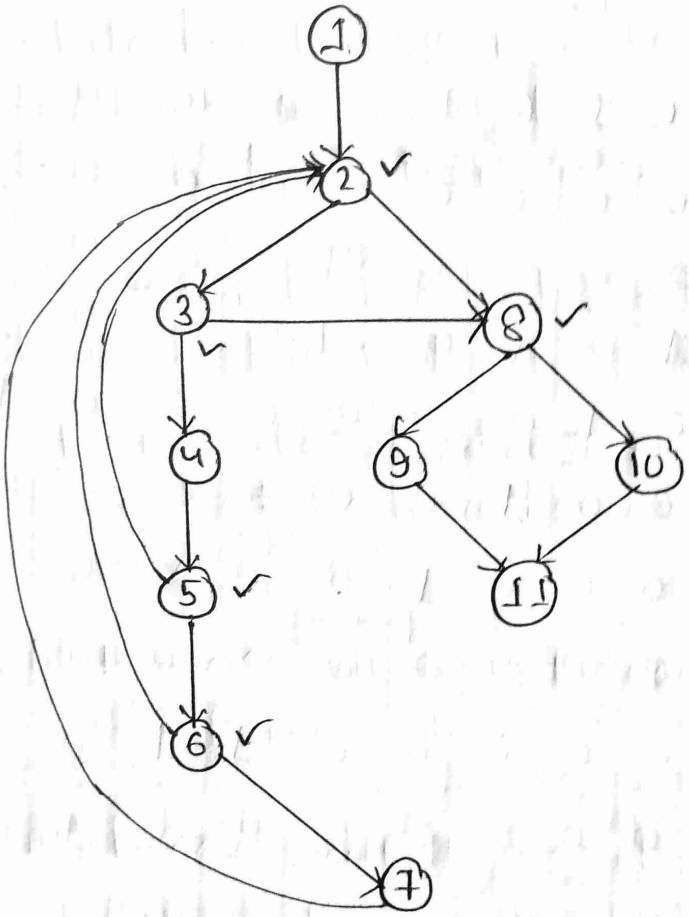
If 

while 

Until 

case 

Example:-



☐ Cyclomatic Complexity:-

(1) $V(G) = E - N + 2$

(2) $V(G) = P + 1$

$V(G) \longrightarrow$ No. of independent paths

E = Edges
N = Nodes
P = No of nodes with multiple outgoing edges
↳ Predicate nodes

Example-

For figure (1) $V(G) = 15 - 11 + 2 = 6$

(2) $V(G) = 5 + 1 = 6$

∴ figure has 6 independent paths.

Graph Matrices:-

	1	2	3	4	5	6	7	8	9	10	11	
1	0	1	0	0	0	0	0	0	0	0	0	1-1=0
2	0	0	1	0	0	0	0	1	0	0	0	2-1=1
3	0	0	0	1	0	0	0	1	0	0	0	1-1=0
4	0	0	0	0	1	0	0	0	0	0	0	1-1=0
5	0	1	0	0	0	1	0	0	0	0	0	2-1=1
6	0	1	0	0	0	0	1	0	0	0	0	2-1=1
7	0	1	0	0	0	0	0	0	0	0	0	1-1=0
8	0	0	0	0	0	0	0	0	1	1	0	2-1=1
9	0	0	0	0	0	0	0	0	0	0	1	1-1=0
10	0	0	0	0	0	0	0	0	0	0	1	1-1=0
11	0	0	0	0	0	0	0	0	0	0	0	0 =0

5

↳ Predicate nodes

Basis Path Testing

flow graph

↓

Cyclomatic Complexity

↓

Independent path

↓

Generate test cases

↗ Behavioral testing

Black-Box Testing:-

- Black-box testing focuses on the functional requirements of the software.
- It enables the soft. engineer to derive sets of input conditions that will fully exercise all functional requirements for a program.
- It is not an alternative to white box testing, rat a complementary approach that is likely to uncover a different class of errors.
- Black-box testing attempts to find errors in the following categories -
  (i) Incorrect or missing functions.
  (ii) Interface errors.
  (iii) Errors in data structures or external database access.
  (iv) Behaviour or performance errors
  (v) Initialization and termination errors.

Graph Based Testing Methods:-

Nodes ⟶ represent Objects we need to test
Links ⟶ represent relationship between objects.
↳ operations/methods/features

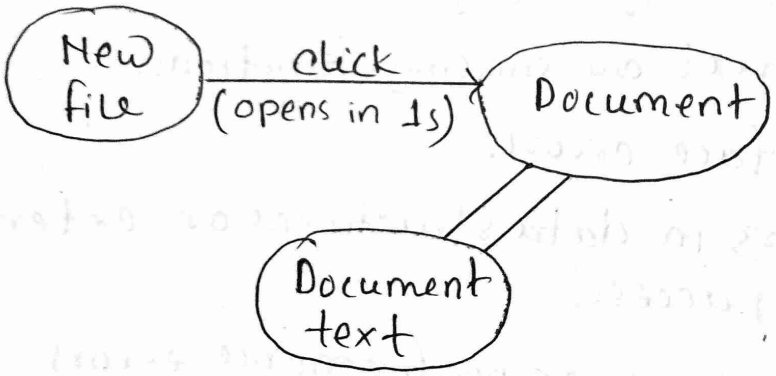
Node Weights → Describes the properties of a node

Link Weights → Describe characteristics of a link.

Example –

File
- New
- Open
- Close

Document
- Start type
- Delete



Directed Link → Indicates that a relationship moves in only one direction

Bidirectional Link → Relationship applies in both direction

Parallel Links → A number of different relationships are established between graph nodes.

Equivalence Partitioning:-

- A black-box testing method that divides the input domain of a program into classes of data from which test cases can be derived.

(1) Range:-

↳ One valid, two invalid

Ex-

let the range is 0-100

Invalid	valid	Invalid
-1	0	101

(2) Specific value:-

↳ One valid, two invalid

Ex- let the value is 100

Invalid	valid	Invalid
99	100	101

(3) Set:-

↳ One valid, one invalid

Ex- Check, deposit, bill pay etc.

(4) Boolean:-

↳ One valid, one invalid

Ex-

username exists, password exists → valid

username doesn't exist → Invalid

Boundary value Analysis:-

↳ Testing with minimum values of input and output.

Let the rang is $[a, b]$

we have to test with -

$a-1,\ a+1,\ b+1,\ b-1$

Testing objectives:-

(1) Testing is the process of executing a program with the intent of finding an error.

(2) A good test case is one that has a high probability of finding an undiscovered error.

(3) A successful test is one that uncovers an as-yet-undiscovered error.

Testing Principles:-

(1) All tests should be traceable to customer requirements.

(2) Tests should be planned long before testing begins.

(3) The pareto principle applies to software testing.

(4) Testing should begin "in the small" and progress towards "in the large".

(5) Exhaustive testing is not possible.

(6) Testing should be conducted by an independent third party.

Chapter-18

Verification vs. Validation:-

- Verification refers to the set of tasks that ensure that software correctly implements a specific function
  - → Are we building the product right?

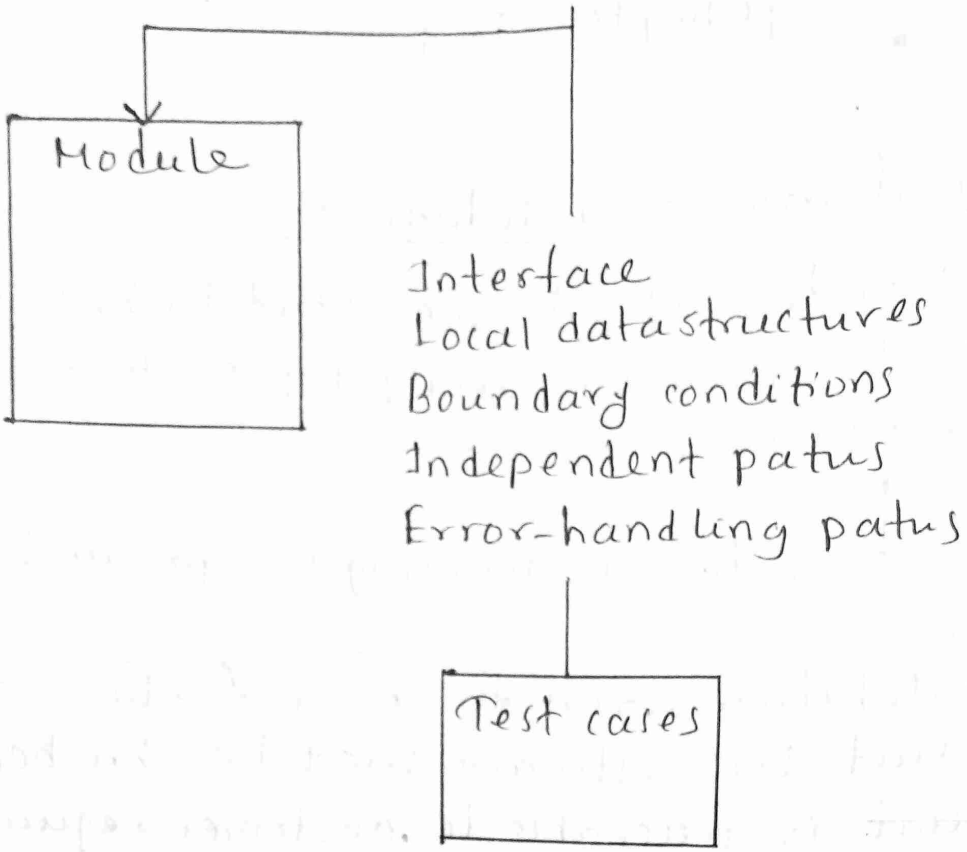
- Validation refers to a set of tasks that ensure that the software that has ~~bee~~ been built ~~that~~ is traceable to customer requirements.
  - → Are we building the right product?

Software testing strategy:-

Unit testing ⟶ Unit testing
Integration testing ⟶ Integration testing
Validation testing ⟶ Validation testing
System testing ⟶ System testing

Unit Testing:-

- Unit testing focuses verification efforts on the smallest unit of software design — the software component or ~~do~~ module.

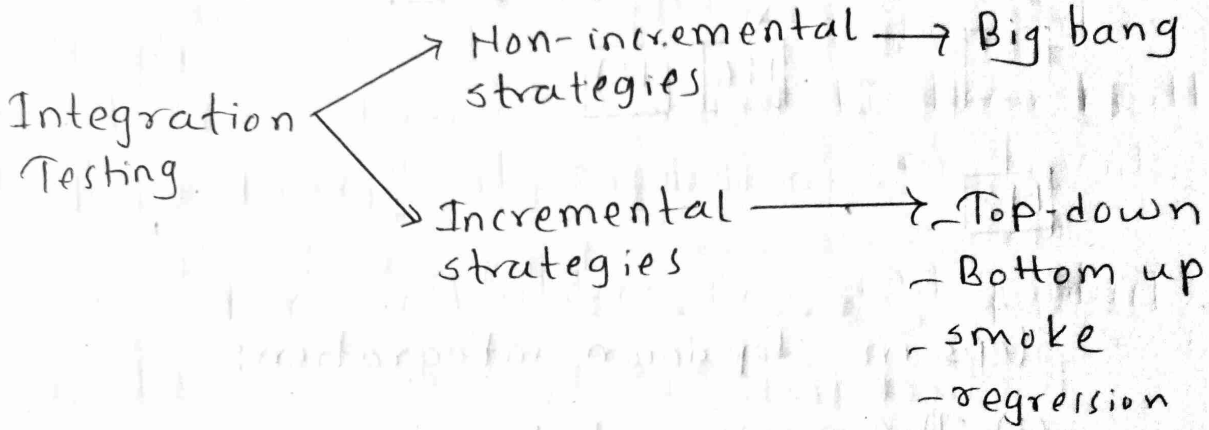


- The module interface is tested to ensure that information properly flows into and out of the program unit under test.
- Local data structures are examined to ensure that data stored temporarily maintains its integrity during all steps in an algorithms execution.
- All independent paths through the control structure are exercised to ensure that all statements in a module have been executed at least once.
- Boundary conditions are tested to ensure that the module operates properly at boundaries established to limit or restrict processing

- Finally, all error handling paths are tested.

Integration Testing:-

- A phase in software testing in which individual software modules are combined and tested as a group.



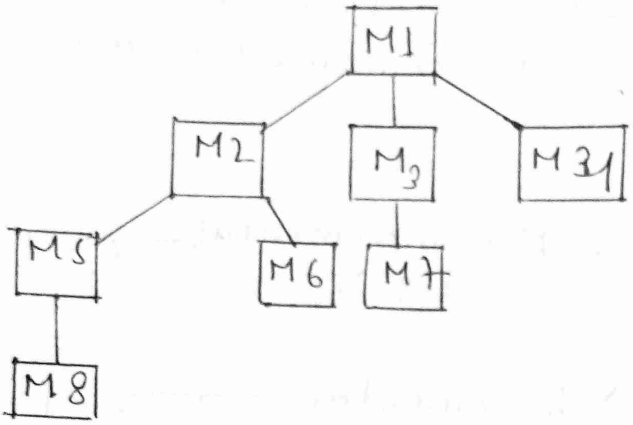
Big-bang approach:-

- All components are combined in advance. The entire program is tested as a whole. And chaos usually result!
- A set of errors is encountered. Correction is difficult because isolation of causes is complicated by the vast expanse of the entire program.
- Once these errors are corrected, new ones appear and the process continues.

→ When we know the function
যখন জানি কী করা হবে

☐ Top-down integration:—

- Modules are integrated by moving downward through the control hierarchy, beginning with the main control module



- BFS / DFS approach

- Steps in Top down integration:

(1) The main control module is used as a test driver and the stubs are substituted for all components directly subordinating to the main control module.

(2) Depending on the integration approach selected, subordinate stubs are ~~and~~ ~~selected~~ replaced one at a time with actual components.

(3) Tests are conducted as each component is integrated.

(4) On completion of each set of tests, another stub is replaced with the real component.

(5) Regression testing may be conducted to ensure the new errors have not been introduced.

→ এখন আবার না কী করা করবে?

☐ Bottom-Up integration:—

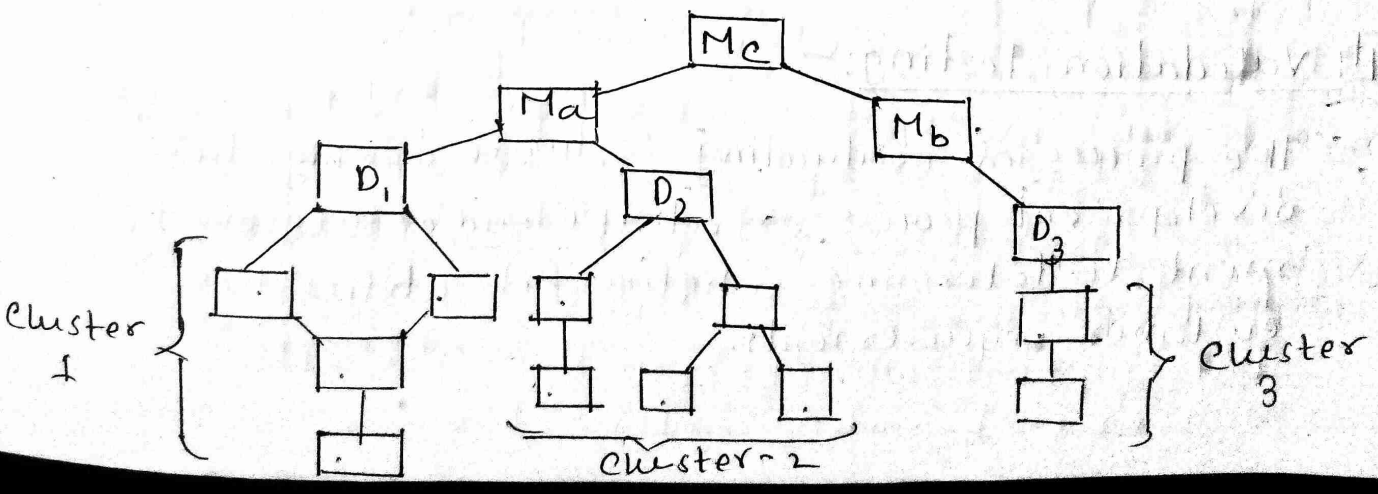
- Begins construction and testing with components at the lowest levels in the program structure.
- Steps in bottom-up approach—

(1) Low level components are combined into clusters that perform a specific software subfunction.

(2) A driver is written to coordinate test case input and output.

(3) The cluster is tested.

(4) Drivers are removed and clusters are combined moving upward in the program structure.



Regression Test:-

- Each time a new module is added as a part of integration testing, the software changes. Regression testing is the re-execution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects.

Smoke testing:-

- An integration testing approach that is commonly used when product software is developed.
- It check for-
  (1) whether the build has missing files or not
  (2) Uncover "show stopper" errors.
  (3) Check for errors on daily basis.

Validation Testing:-

- The process of evaluating software during the development process or at the end of the development to determine whether it satifies the customer requirements.

Alpha and Beta testing:-

- Most software product builders use a process called alpha and beta testing to uncover error that only the end users seem able to find.
- The alpha test is conducted at the developer's site by a representative group of end users. The software is used in a natural settings with the developer "looking over the shoulder" of the users and recording errors and usage problems. Alpha tests are conducted in a controlled environment.
- The beta test is conducted at one or more end-user sites. The developer generally is not present. The customer records all problems that are encounted during beta testing and reports these to developers.

System Testing:-

- System testing is actually a set of different tests whose primary ~~te~~ purpose is to fully exercise the computer based system.

Recovery Testing:-

- Recovery testing is a system test that forces the software to fail in a variety of ways and verifies that recovery is properly performed.

Security Testing:-

- Security testing attempts to verify that protection mechanism built into a system will, infact, protect it from improper penetration.

- The tester may attempt to acquire passwords through external clerical means, may attack the system with custom software designed to break down any defenses that have been constructed.

- Given enough time and resources, good security testing will ultimately penetrate a system. The ~~role~~ role of the system designer is to make penetration cost more than the value of the information that will be obtained.

stress testing:-

- A software testing activity that determines the robustness of the software by testing beyond the limits of normal operation.
- Normally important for 'mission critical' software. But, used for all types of software.

~~Emphasis on~~

- Put emphasis on robustness, availability and error handling under a heavy load,

than on what would be considered correct behaviour under normal circumstances.

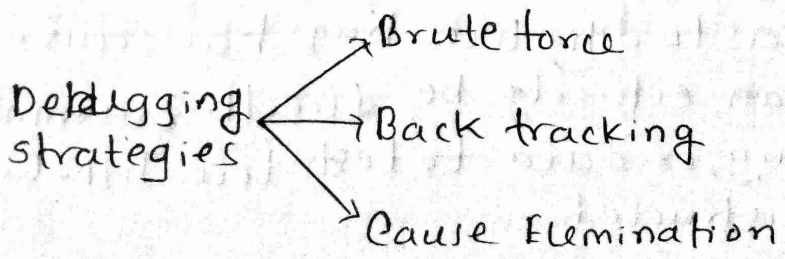
Performance Testing:-

- It is designed to test run-time performance of software within the context of an integrated system.

- Performance testing occurs throughout all steps in the testing process. Even at the unit level. However it is not until all system elements are fully integrated that the true performance of a system can be ascertained.

Debugging:-

- Debugging is the process of finding and resolving defects or problems within a computer program that prevents correct operation of computer software or a system.



Brute force:-

- This involves the developer manually searching through stack traces, memory dumps, log files and so on for tracing the error.
- Extra output statements (printf()), in addition to break points, are often added to the code in order to examine what the software is doing at every step.
- Most common and least efficient method. Apply when all method fails.

Back-tracking:-

- The developer begin with the code that immediately produces the observable error. The developer then backtracks through the execution paths, looking for the cause.
- Unfortunately, as the number of source lines increases, the number of potential backward paths may become unmanagebly large.

Cause elemination:-

- In this method, the developer develops hypothesis as to why the bug has occured. The code can either be directly examined for the bug, or data to test the hypothesis can be constructed.

- This method can often result in the shortest debug times, although it does rely on the developer understanding the software well.

Correcting the error:-

- Once a bug has been found, it must be corrected. But, as we have already noted, the correction of a bug can introduce other errors. and therefore do more harms than good.
- Van ~~Hake~~ Vleck suggests three simple questions that should be asked before making the correction-

(1) Is the cause of the bug reproduced in another part of the program? In many situations, a program defect is caused by an ~~erroneous~~ erroneous pattern of logic that may be reproduced elsewhere. Explicit consideration of the logical pattern may result in the discovery of other errors.

(2) where next bug might be introduced by the fix I'm about to make? If the correction is to be made in a highly coupled section of the program, special care must be taken when any change is made.

(3) What could we have done to prevent this bug in the first place?

Chapter-10

System:-

Computer-based System:-

- A set of arrangement of elements that are organized to accomplish some predefined goal by processing information.
- The goal may be to support some business function or to develop a product that can be sold to generate business revenue.

Elements of computer-based system:-

- To accomplish the goal, a computer based system makes use of a variety of system elements -

(1) Software:-

- Computer programs
- Data structures
- Related documentation

(2) Hardware:-

- Electronic devices that provide computing capability.
- The inter connectivity devices that enable the flow of data.
- Electromechanical devices that that provide external world function.

(3) People:-

- Users and operators of hardware and softwar

(4) Database:-

- A large, organized collection of informati that is accessed via software.

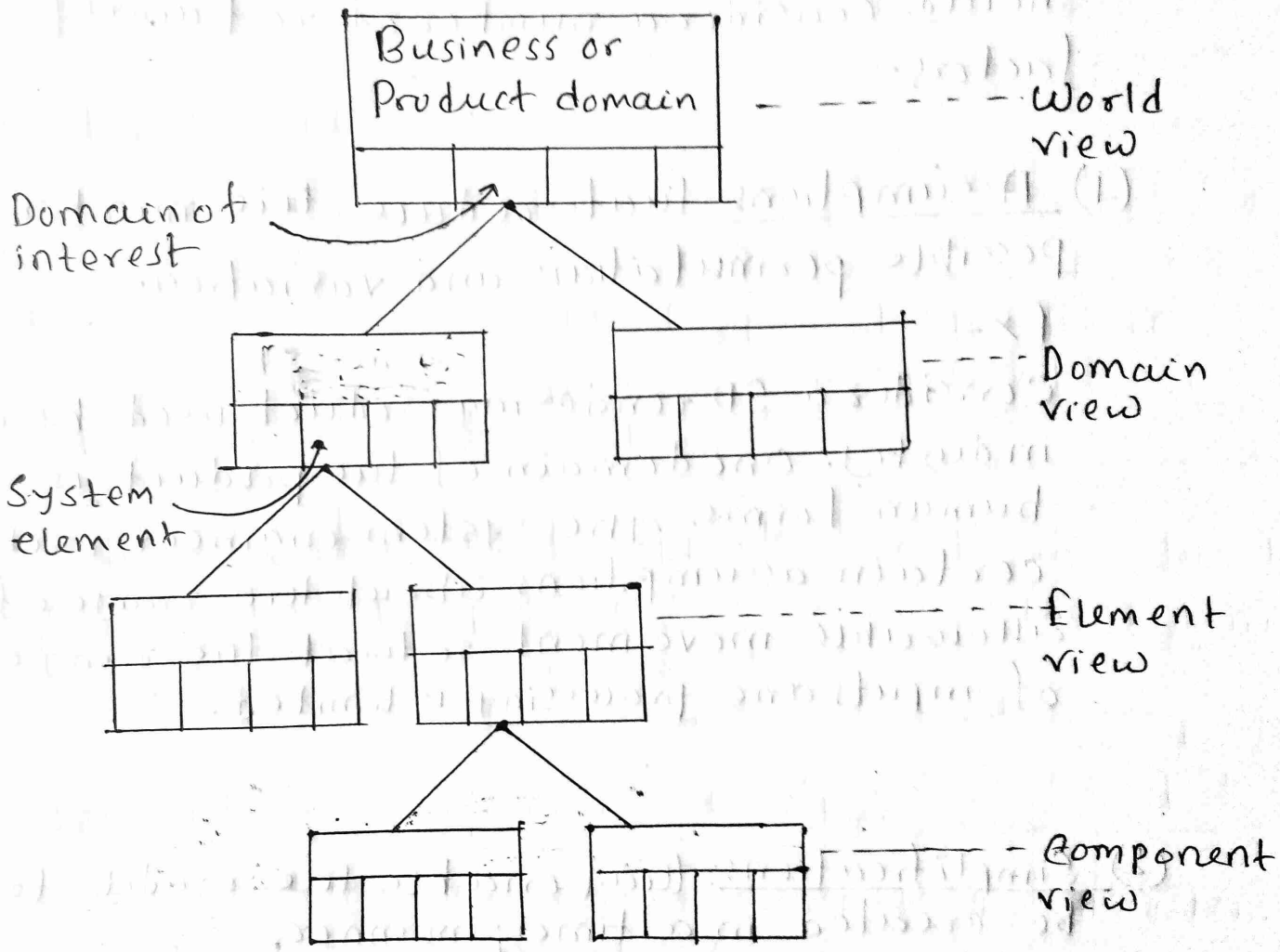
(5) Documentation:-

- Descriptive information that portrays the use and operation of the system.

(6) Procedures:-

- The steps that define the specific use of each system element.

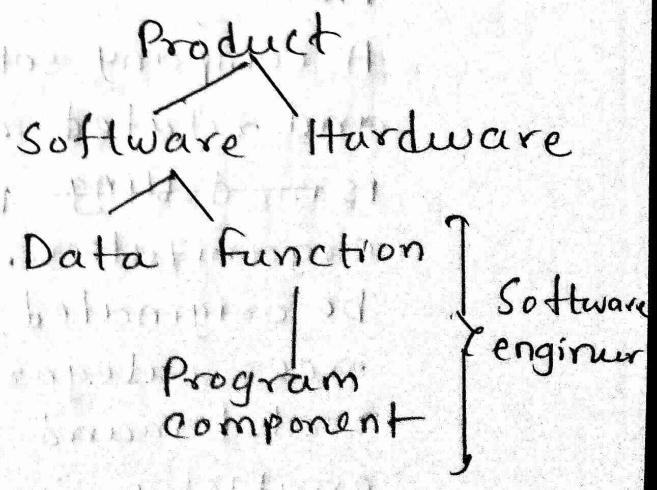
The system Design Hierarchy:



$$WV = \{ D_1, D_2, D_3, \ldots, D_n \}$$

$$D_i = \{ E_1, E_2, E_3, \ldots, E_m \}$$

$$E_j = \{ e_1, e_2, e_3, \ldots, e_k \}$$



System Modeling :-

- To construct a system model, the engineer should consider a number of restraining factors-

(1) Assumptions that reduce the number of possible permutations and variation.

Ex-

Consider a 3D rendering product used by an industry. One domain of the product is 3D human forms. The system engineer makes certain assumptions about the range of allowable movement so that the range of inputs and processing is limited.

(2) Simplifications: that enable the model to be created in a timely manner.

Ex-

A company sells and services copiers, faxes and related equipments. The system engineer is ~~modelling~~ modeling the needs of service organization. Although the service order can be originated from many sources, the engineer categorites only two sources — internal demand and external request. This enables a simplified partitioning.

(3) Limitations that help to bound the system:-
Ex-
An aircraft electronic system is being modeled for a next generation aircraft. Since, the aircraft will have a two engine design, the monitoring domain for propulsion will be modeled to accommodate a max. of two engines.

(4) Constrains that will guide the manner in which the model is created and the approach taken when the model is implemented.

↳ Model তৈরী ও Implementation এর চলার কিছু বাধ্যবাধকতা আরোপ করবে।

Ex-
The 3D model generating company discussed before, uses a single GH-based processor. The computational complexity of the problems must be constrained to fit within the processing bounds imposed by the processor.

(5) Preferences: that indicate the preferred architecture ~~of~~ for an data, functions and technology.